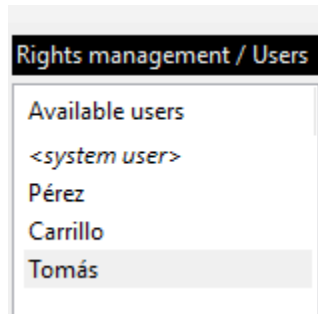


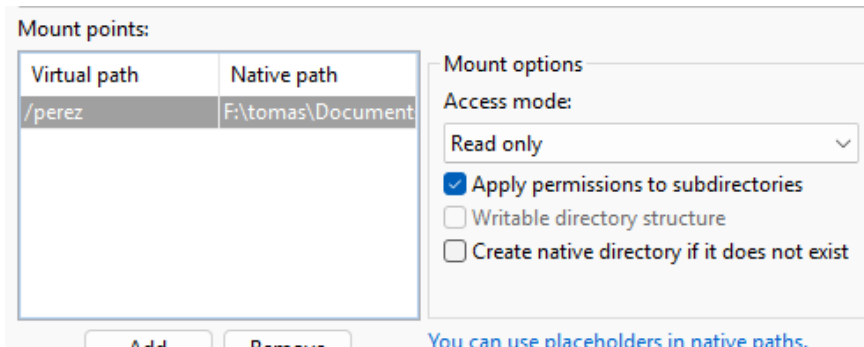
Act04. Generación de Servicios en Red

PARTE 1. FTP

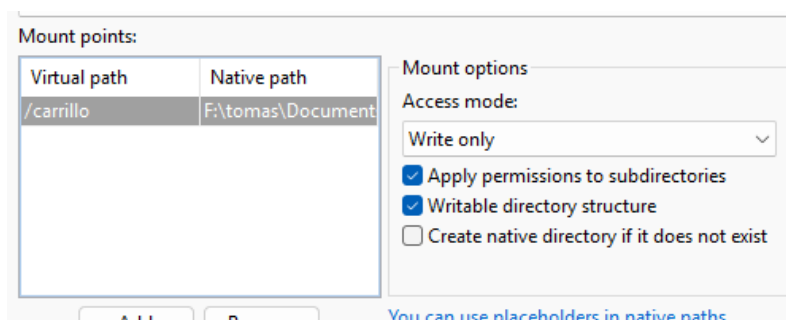
Lo primero de todo ha sido instalar FileZilla Server en mi ordenador personal. Lo siguiente ha sido realizar la creación de los 3 usuarios asignando las rutas y los permisos indicados en la tarea:



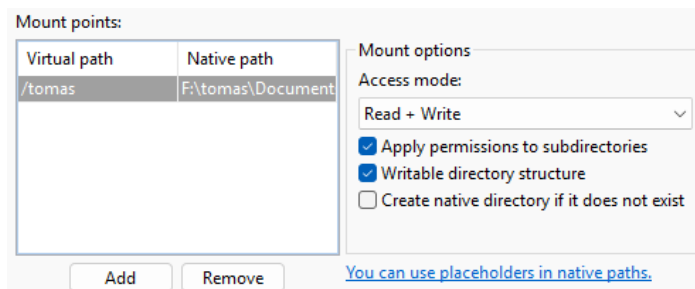
Pérez (solo lectura):



Carrillo (solo escritura):



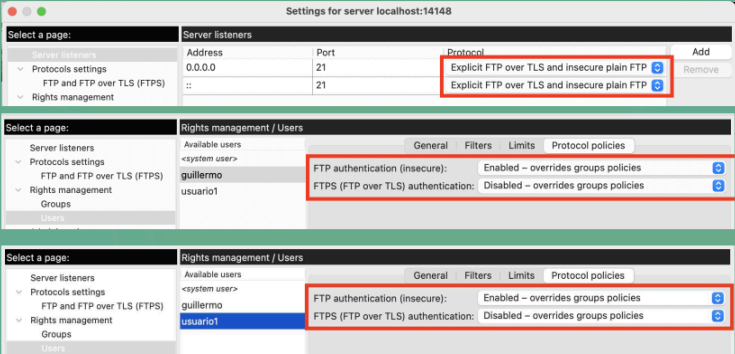
Tomás (ambos permisos):



La ruta nativa es la misma en los tres usuarios (“F:\tomas\Documents\PSP-Act04\datos”).

Además, he configurado la seguridad de FileZilla Server tal cual aparece en la página 20 de los apuntes.

► Puesto que vamos a trabajar en texto plano tenemos que cambiar dos cuestiones en la seguridad de FileZilla Server (la segunda se debe realizar para cada usuario)



Antes de hacer los ejercicios, he creado las variables de entorno para cada Usuario y añadido las dependencias necesarias de este RA:

```
C:\> Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.7623]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\tomas>setx FTP_SERVER localhost
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>setx FTP_PORT 21
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>setx FTP_USERTOMAS Tomás
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>setx FTP_USERPEREZ Pérez
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>setx FTP_USERCARRILLO Carrillo
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>setx FTP_PASSWORD alumnoDAM
CORRECTO: se guardó el valor especificado.

C:\Users\tomas>|
```

NOTA: TODO LO QUE APARECE CON TILDES HA SIDO MODIFICADO A SIN TILDES POR MOTIVOS DE ERRORES AL LEER DICHAS VARIABLES. (Tomás → Tomas).

```
<dependencies>
  <!-- Source: https://mvnrepository.com/artifact/commons-net/commons-net -->
  <dependency>
    <groupId>commons-net</groupId>
    <artifactId>commons-net</artifactId>
    <version>3.12.0</version>
    <scope>compile</scope>
  </dependency>
  <!-- Source: https://mvnrepository.com/artifact/org.eclipse.angus/angus-mail -->
  <dependency>
    <groupId>org.eclipse.angus</groupId>
    <artifactId>angus-mail</artifactId>
    <version>2.0.5</version>
    <scope>compile</scope>
  </dependency>
</dependencies>
```

1. Conecta al servidor y realiza el login utilizando variables de entorno para el usuario y la contraseña.

```
public class Tomas { new *

    public static void main(String[] args) { new *

        FTPClient ftp = conexion();

    }

    private static FTPClient conexion(){ 1usage new *
        try {

            String server = System.getenv( name: "FTP_SERVER");
            int port = Integer.parseInt(System.getenv( name: "FTP_PORT"));
            String username = System.getenv( name: "FTP_USERTOMAS"); //Esto es lo que cambiará
            String password = System.getenv( name: "FTP_PASSWORD");

            FTPClient ftp = new FTPClient();

            ftp.connect(server,port);
            int codigo = ftp.getReplyCode();
            if (!FTPReply.isPositiveCompletion(codigo)) {
                ftp.disconnect();
            }

            boolean login = ftp.login(username,password);
            System.out.println("Se ha conectado el usuario " + username + ": " + login);

            if (!login) {
                ftp.enterLocalPassiveMode();
                ftp.disconnect();
                return null;
            }

            return ftp;
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Cada usuario tiene una clase y para el ejercicio1 el código es el mismo lo único que cambia es la variable de username cambiando la variable de entorno para cada usuario como se ve al principio en la captura de la terminal. Por motivos de espacio he decidido solo mostrar la clase Tomas.

El output de la consola es el siguiente:

```
Se ha conectado el usuario Tomas: true
```

```
Se ha conectado el usuario Perez: true
```

```
Se ha conectado el usuario Carrillo: true
```

2. Muestra por consola la ruta del directorio de trabajo actual en el servidor.

```
try {  
    System.out.println("Directorio de trabajo: " + ftp.printWorkingDirectory());  
} catch (IOException e) {  
    throw new RuntimeException(e);  
}
```

```
Se ha conectado el usuario Carrillo: true  
Directorio de trabajo: /
```

```
Se ha conectado el usuario Tomas: true  
Directorio de trabajo: /
```

```
Se ha conectado el usuario Perez: true  
Directorio de trabajo: /
```

3. Intenta cambiarte a un directorio de otro usuario. Por ejemplo, si estás conectado con el usuario apellido1 (palazon) intenta acceder al directorio /cano.

Voy a hacer la prueba intentando conectar al directorio /carrillo desde el usuario Tomas.

```
try {  
    System.out.println("Directorio de trabajo: " + ftp.printWorkingDirectory());  
  
    ftp.changeWorkingDirectory( path: "/carrillo");  
    System.out.println("Directorio de trabajo: " + ftp.printWorkingDirectory());  
} catch (IOException e) {  
    throw new RuntimeException(e);  
}
```

```
Se ha conectado el usuario Tomas: true  
Directorio de trabajo: /  
Directorio de trabajo: /
```

4. Cámbiate al directorio virtual que tiene dicho usuario.

```
try {  
    System.out.println("Directorio de trabajo: " + ftp.printWorkingDirectory());  
  
    ftp.changeWorkingDirectory( path: "/tomas");  
    System.out.println("Directorio de trabajo: " + ftp.printWorkingDirectory());  
} catch (IOException e) {  
    throw new RuntimeException(e);  
}
```

```
Se ha conectado el usuario Tomas: true  
Directorio de trabajo: /  
Directorio de trabajo: /tomas
```

5. Crea una carpeta en el servidor que se llamará mdUsuario (Por ejemplo, si estás con el usuario palazon la carpeta se llamará mdPalazon) y otra carpeta que se llamará fotosUsuario. Para cada usuario se crearán carpetas distintas.

```
private static ArrayList<String> creacionCarpetas(FTPClient ftp){ 1 usage new *

    ArrayList<String> carpetas = new ArrayList<>();

    try {
        String username = System.getenv( name: "FTP_USERTOMAS");

        String carpetaMd = "md" + username;
        String carpetaFotos = "fotos" + username;

        carpetas.add(carpetaMd);
        carpetas.add(carpetaFotos);

        boolean creada1 = ftp.makeDirectory(carpetaMd);
        boolean creada2 = ftp.makeDirectory(carpetaFotos);

        if (creada1) {
            System.out.println("Carpeta creada con éxito: " + carpetaMd);
        } else {
            System.out.println("No se pudo crear la carpeta: " + carpetaMd + " (quizás ya existe o falta permiso)");
        }

        if (creada2) {
            System.out.println("Carpeta creada con éxito: " + carpetaFotos);
        } else {
            System.out.println("No se pudo crear la carpeta: " + carpetaFotos);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

```
Se ha conectado el usuario Tomas: true
Directorio de trabajo: /
Directorio de trabajo: /tomas
Carpeta creada con éxito: mdTomas
Carpeta creada con éxito: fotosTomas
```

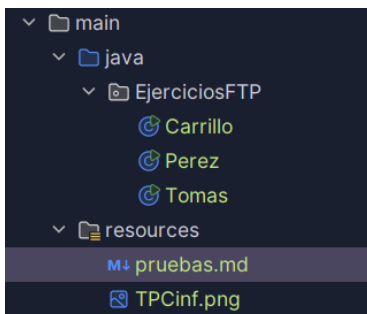
El código es el mismo para todos los usuarios.

```
Se ha conectado el usuario Perez: true
Directorio de trabajo: /
Directorio de trabajo: /perez
No se pudo crear la carpeta: mdPerez (quizás ya existe o falta permiso)
No se pudo crear la carpeta: fotosPerez
```

El usuario Perez no deja ya que solo tiene permisos de lectura.

```
Se ha conectado el usuario Carrillo: true
Directorio de trabajo: /
Directorio de trabajo: /carrillo
Carpeta creada con éxito: mdCarrillo
Carpeta creada con éxito: fotosCarrillo
```

6. Dentro de la carpeta src/main/resources del proyecto debes copiar un fichero .md y un fichero .jpg (esto se hace manual, no con código Java y solamente se debe hacer una vez y no para cada usuario). Debes hacer la subida de los mismos al servidor a las carpetas correspondientes que has creado en el apartado anterior (cada uno va a una carpeta distinta) teniendo en cuenta el modo de transferencia correspondiente.



```
public static void main(String[] args) { new *
    try {
        FTPClient ftp = conexion();

        if (ftp == null) return; //Por si falla la conex

        mostrarDirectorioActual(ftp);

        ArrayList<String> carpetasCreadas = creacionCarpetas(ftp);

        String carpetaMD = carpetasCreadas.get(0);
        String carpetaFotos = carpetasCreadas.get(1);

        String rutaFicheroMD = "src/main/resources/pruebas.md";
        String rutaFicheroPNG = "src/main/resources/TPCinf.png";

        boolean subidoASCII = subirFicheroASCII(ftp, rutaFicheroMD, nombreServidor: "textoMDPRUEBAS.md", carpetaMD);

        if (subidoASCII) {
            System.out.println("Fichero de texto subido con éxito.");
        } else {
            System.out.println("Error al subir fichero de texto.");
        }

        boolean subidoPNG = subirFicheroBinario(ftp, rutaFicheroPNG, nombreServidor: "imagenJPGPRUEBAS.png", carpetaFotos);

        if (subidoPNG) {
            System.out.println("Fichero de imagen subido con éxito.");
        } else {
            System.out.println("Error al subir fichero de imagen.");
        }

    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

```
private static boolean subirFicheroBinario(FTPClient ftp, String rutaAbsolutaNombre) {

    ftp.changeWorkingDirectory(path: "/Tomas");
    System.out.println("DIRECTORIO" + ftp.printWorkingDirectory());

    ftp.changeWorkingDirectory(carpetasDestino);
    System.out.println("DIRECTORIO ACTUAL" + ftp.printWorkingDirectory());

    ftp.setFileType(FTPClient.BINARY_FILE_TYPE);
    File f = new File(rutaAbsolutaNombre);
    InputStream in = new FileInputStream(f);
    OutputStream out = ftp.storeFileStream(nombreServidor);

    byte[] buffer = new byte[4096];
    int read;
    while ((read = in.read(buffer)) != -1) {
        out.write(buffer, 0, read);
    }
    in.close();
    out.close();
    return ftp.completePendingCommand();
}

private static boolean subirFicheroASCII(FTPClient ftp, String rutaAbsolutaNombre) {

    ftp.changeWorkingDirectory(path: "/Tomas");
    System.out.println("DIRECTORIO" + ftp.printWorkingDirectory());

    ftp.changeWorkingDirectory(carpetasDestino);
    System.out.println("DIRECTORIO ACTUAL" + ftp.printWorkingDirectory());

    ftp.setFileType(FTPClient.ASCII_FILE_TYPE);

    File f = new File(rutaAbsolutaNombre);

    InputStream in = new FileInputStream(f);
    boolean upload = ftp.storeFile(nombreServidor, in);

    in.close();
    return upload;
}
```

```
Se ha conectado el usuario Tomas: true
Directorio de trabajo: /
Directorio de trabajo: /tomas
No se pudo crear la carpeta: mdTomas (quizás ya existe o falta permiso)
No se pudo crear la carpeta: fotosTomas
DIRECTORIO/Tomas
DIRECTORIO ACTUAL/Tomas/mdTomas
Fichero de texto subido con éxito.
DIRECTORIO/Tomas
DIRECTORIO ACTUAL/Tomas/fotosTomas
Fichero de imagen subido con éxito.

Process finished with exit code 0
```

El usuario Tomas da error (subrayado en azul) porque ya creé antes las carpetas.

Tomás Pérez Carrillo
2º DAM - Programación Servicios y Procesos

```
Se ha conectado el usuario Perez: true
Directorio de trabajo: /
Directorio de trabajo: /perez
No se pudo crear la carpeta: mdPerez (quizás ya existe o falta permiso)
No se pudo crear la carpeta: fotosPerez
DIRECTORIO/Perez
DIRECTORIO ACTUAL/Perez
Error al subir fichero de texto.
DIRECTORIO/perez
DIRECTORIO ACTUAL/perez
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.io.OutputStream.write(byte[], int, int)" because "out" is null
    at EjerciciosFTP.Perez.subirFicheroBinario(Perez.java:67)
    at EjerciciosFTP.Perez.main(Perez.java:37)

Process finished with exit code 1
```

El usuario Perez da error porque solo tiene permisos de lectura.

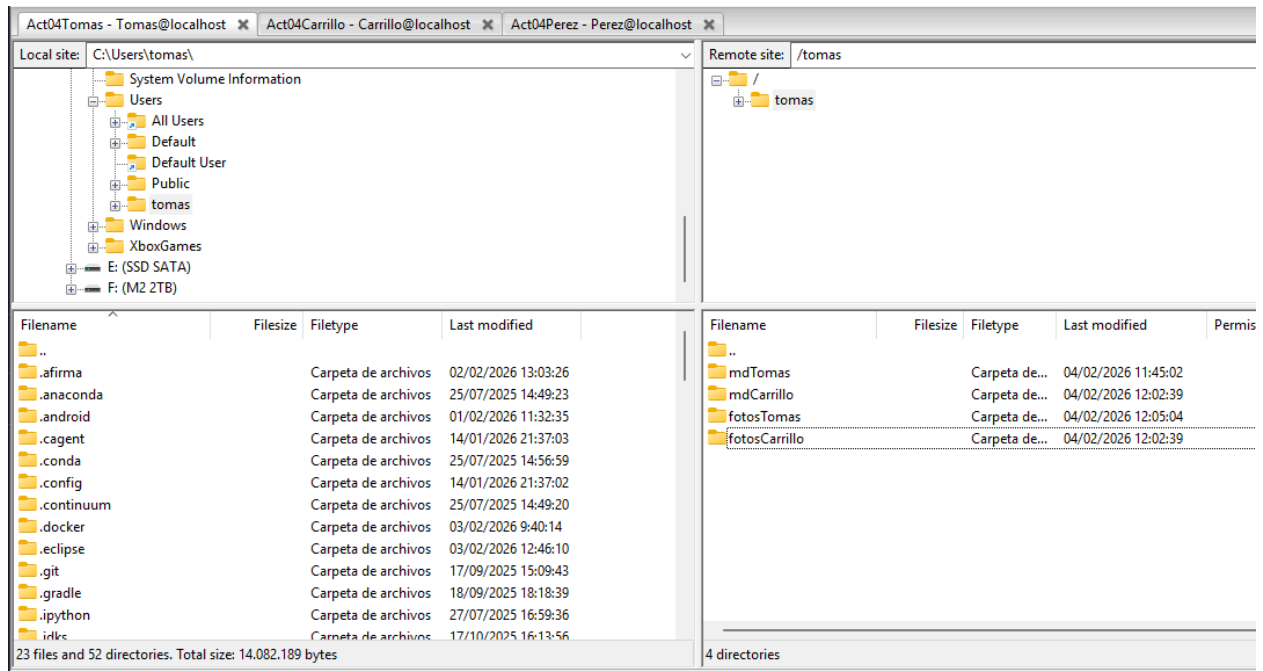
```
C:\Users\tomas\.jdk\openjdk-24\bin\java.exe "-javaagent:C:\Program Files\JetBr
Se ha conectado el usuario Carrillo: true
Directorio de trabajo: /
Directorio de trabajo: /carrillo
No se pudo crear la carpeta: mdCarrillo (quizás ya existe o falta permiso)
No se pudo crear la carpeta: fotosCarrillo
DIRECTORIO/carrillo
DIRECTORIO ACTUAL/carrillo/mdCarrillo
Fichero de texto subido con éxito.
DIRECTORIO/carrillo
DIRECTORIO ACTUAL/carrillo/fotosCarrillo
Fichero de imagen subido con éxito.

Process finished with exit code 0
```

El usuario Carrillo da error (subrayado en azul) porque ya cree las carpetas antes pero funciona al igual que Tomas.

Evidencia de este ejercicio en FileZilla Client:

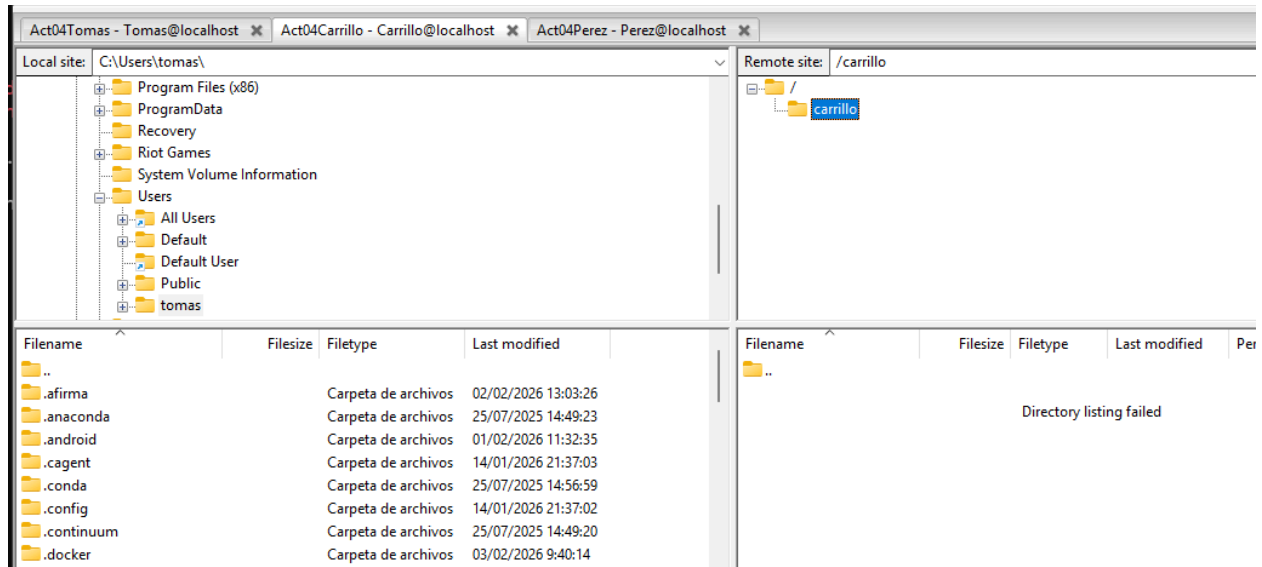
Usuario Tomas tiene permiso para leer y escribir por lo que su sesión en el cliente muestra esto:



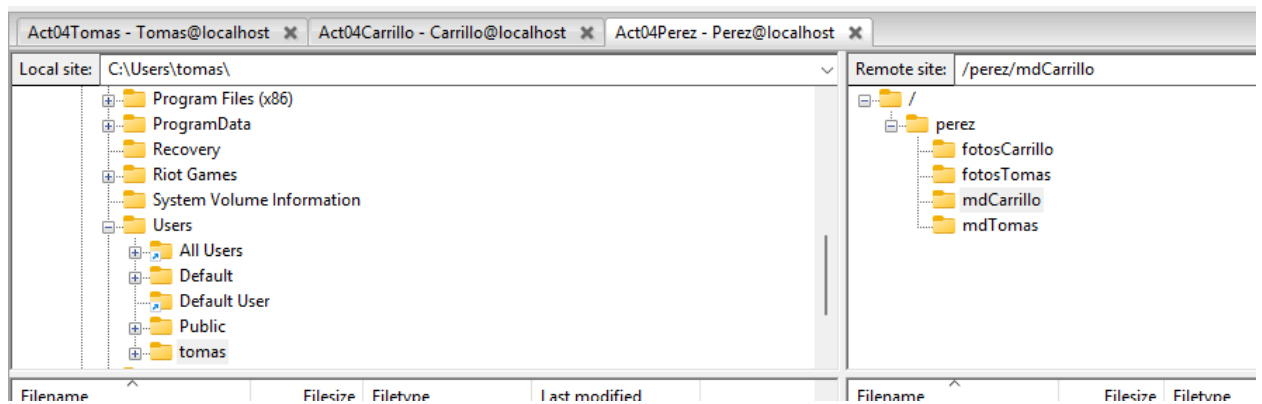
Dentro de cada carpeta está el archivo correspondiente. No se si así se ve mejor en vez de tanta captura pero el contenido es el mismo tanto en el cliente como en local.

```
PS F:\tomas\Documents\PSP-Act04> tree /f
Listado de rutas de carpetas para el volumen M2 2TB
El número de serie del volumen es 36FE-CF26
F:..
├── datos
│   ├── fotosCarrillo
│   │   └── imagenJPGPRUEBAS.png
│   ├── fotosTomas
│   │   └── imagenJPGPRUEBAS.png
│   ├── mdCarrillo
│   │   └── textoMDPRUEBAS.md
│   └── mdTomas
│       └── textoMDPRUEBAS.md
```

Sesión de Carrillo:

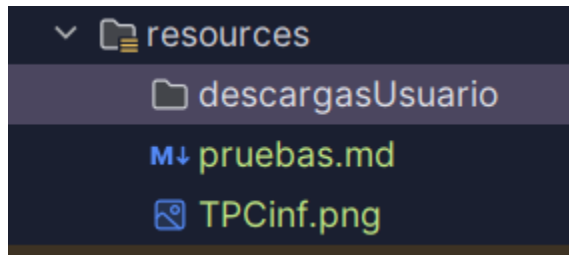


Carrillo no ve nada porque solo tiene escritura.



Perez lo ve todo pero no puede escribir (por eso errores en el ide al ejecutar).

7. Crea en el proyecto local, dentro del proyecto en la carpeta src/main/resources una carpeta que se llame descargasUsuario (puedes hacerlo con código Java, aunque no se hace con el objeto FTPClient ya que se hace en local y no en remoto).



8. Descarga los archivos previamente subidos en la nueva carpeta creados.

Parte del código común a los usuarios:

```
System.out.println("--- INICIANDO DESCARGAS ---");

String destinoMD = "src/main/resources/descargasUsuario/descargado_pruebas.md";
boolean bajadoMD = descargarFichero(ftp, carpetaMD, nombreArchivoEnServidor, "textoMDPRUEBAS.md", destinoMD, FTPClient.ASCII_FILE_TYPE);

if (bajadoMD) System.out.println("MD Descargado correctamente en: " + destinoMD);
else System.out.println("Error descargando MD.");

String destinoPNG = "src/main/resources/descargasUsuario/descargado_imagen.png";
boolean bajadoPNG = descargarFichero(ftp, carpetaFotos, nombreArchivoEnServidor, "imagenJPGPRUEBAS.png", destinoPNG, FTPClient.BINARY_FILE_TYPE);

if (bajadoPNG){
    System.out.println("PNG Descargado correctamente en: " + destinoPNG);
}else{
    System.out.println("Error descargando PNG.");
}

ftp.disconnect();

} catch (IOException e) {
    throw new RuntimeException(e);
}
```

```
private static boolean descargarFichero(FTPClient ftp, String carpetaRemota, String nombreArchivoEnServidor, String rutaLocalDestino) {

    ftp.changeWorkingDirectory(path: "/Tomas");

    ftp.changeWorkingDirectory(carpetaRemota);

    ftp.setFileType(tipoFichero); // En base a parametro recibido por si es md o png

    File archivoLocal = new File(rutaLocalDestino);

    OutputStream output = new FileOutputStream(archivoLocal);
    boolean download = ftp.retrieveFile(nombreArchivoEnServidor, output);
    output.close();
    return download;
}
```



Tomas:

```
--- INICIANDO DESCARGAS ---  
MD Descargado correctamente en: src/main/resources/descargasUsuario/descargado_pruebas.md  
PNG Descargado correctamente en: src/main/resources/descargasUsuario/descargado_imagen.png
```

Perez:

```
--- INICIANDO DESCARGAS ---  
Error descargando MD.  
Error descargando PNG.
```

Para ver este error he tenido que comentar la parte de subir ficheros porque salta excepcion.

Carrillo:

```
--- INICIANDO DESCARGAS ---  
Error descargando MD.  
Error descargando PNG.
```

9. Muestra información sobre los archivos subidos al servidor utilizando los métodos proporcionados por la clase FTPCliente como el tamaño del fichero y otros (puedes utilizar métodos adicionales a los existentes en los apuntes)

```
System.out.println("--- MOSTRANDO INFORMACIÓN DE ARCHIVOS ---");  
  
mostrarFicheros(ftp, carpetaMD, carpetaFotos);
```

```
private static void mostrarFicheros(FTPClient ftp, String carpetaRevisarMD, String carpetaRevisarPNG) { 1 usage new "  
  
    try {  
        ftp.changeWorkingDirectory(path: "//Tomas");//Volvemos a la raiz del usuario  
  
        ftp.changeWorkingDirectory(carpetaRevisarMD);  
  
        System.out.println("FICHEROS DE " + carpetaRevisarMD + " del usuario: " + System.getenv( name: "FTP_USERTOMAS"));  
  
        for (FTPFile fichero: ftp.listFiles()) {  
            System.out.println("Nombre: " + fichero.getName() + " .Tamaño: " + fichero.getSize() + " bytes.");  
  
            // Formatear la fecha para que sea legible  
            SimpleDateFormat formato = new SimpleDateFormat( pattern: "dd/MM/yyyy HH:mm:ss");  
            if (fichero.getTimestamp() != null) {  
                String fecha = formato.format(fichero.getTimestamp().getTime());  
                System.out.println("Fecha Modificación: " + fecha);  
            }  
        }  
  
        System.out.println("=====");  
  
        ftp.changeWorkingDirectory( path: "//Tomas");//Volvemos a la raiz del usuario  
  
        ftp.changeWorkingDirectory(carpetaRevisarPNG);  
  
        System.out.println("FICHEROS DE " + carpetaRevisarPNG + " del usuario: " + System.getenv( name: "FTP_USERTOMAS"));  
  
        for (FTPFile fichero: ftp.listFiles()) {  
            System.out.println("Nombre: " + fichero.getName() + " .Tamaño: " + fichero.getSize() + " bytes.");  
  
            // Formatear la fecha para que sea legible  
            SimpleDateFormat formato = new SimpleDateFormat( pattern: "dd/MM/yyyy HH:mm:ss");  
            if (fichero.getTimestamp() != null) {  
                String fecha = formato.format(fichero.getTimestamp().getTime());  
                System.out.println("Fecha Modificación: " + fecha);  
            }  
        }  
    }  
}
```

```
--- MOSTRANDO INFORMACIÓN DE ARCHIVOS ---  
FICHEROS DE mdTomas del usuario: Tomas  
Nombre: textoMDPRUEBAS.md .Tamaño: 121 bytes.  
Fecha Modificación: 04/02/2026 12:03:00  
=====
```

FICHEROS DE fotosTomas del usuario: Tomas
Nombre: imagenJPGPRUEBAS.png .Tamaño: 90972 bytes.
Fecha Modificación: 04/02/2026 12:03:00

Process finished with exit code 0

Pérez:

```
--- MOSTRANDO INFORMACIÓN DE ARCHIVOS ---  
FICHEROS DE mdPerez del usuario: Perez  
Nombre: fotosCarrillo .Tamaño: 0 bytes.  
Fecha Modificación: 04/02/2026 11:24:00  
Nombre: fotosTomas .Tamaño: 0 bytes.  
Fecha Modificación: 04/02/2026 11:20:00  
Nombre: mdCarrillo .Tamaño: 0 bytes.  
Fecha Modificación: 04/02/2026 11:24:00  
Nombre: mdTomas .Tamaño: 0 bytes.  
Fecha Modificación: 04/02/2026 11:20:00  
=====
```

FICHEROS DE fotosPerez del usuario: Perez
Nombre: fotosCarrillo .Tamaño: 0 bytes.
Fecha Modificación: 04/02/2026 11:24:00
Nombre: fotosTomas .Tamaño: 0 bytes.
Fecha Modificación: 04/02/2026 11:20:00
Nombre: mdCarrillo .Tamaño: 0 bytes.
Fecha Modificación: 04/02/2026 11:24:00
Nombre: mdTomas .Tamaño: 0 bytes.
Fecha Modificación: 04/02/2026 11:20:00

Usando el mismo código que en Tomas parece que Perez solo puede ver las carpetas y no el contenido de las mismas.

Carrillo:

```
--- MOSTRANDO INFORMACIÓN DE ARCHIVOS ---  
FICHEROS DE mdCarrillo del usuario: Carrillo  
=====  
FICHEROS DE fotosCarrillo del usuario: Carrillo
```

No muestra nada porque solo tiene permiso de escritura.

10. Crea una nueva carpeta para dicho usuario que se llame pruebaUsuario

Para esta parte he modificado el metodo de creacionCarpetas() en cada usuario y ya.

```
private static ArrayList<String> creacionCarpetas(FTPClient ftp){ 1usage new *  
  
    ArrayList<String> carpetas = new ArrayList<>();  
  
    try {  
        String username = System.getenv( "name", "FTP_USERTOMAS");  
  
        String carpetaMd = "md" + username;  
        String carpetaFotos = "fotos" + username;  
  
        String carpetaPrueba = "prueba" + username;  
  
        carpetas.add(carpetaMd);  
        carpetas.add(carpetaFotos);  
  
        boolean creada1 = ftp.makeDirectory(carpetaMd);  
        boolean creada2 = ftp.makeDirectory(carpetaFotos);  
        boolean creada3 = ftp.makeDirectory(carpetaPrueba);  
  
        if (creada1) {  
            System.out.println("Carpeta creada con éxito: " + carpetaMd);  
        } else {  
            System.out.println("No se pudo crear la carpeta: " + carpetaMd + " (quizás ya existe o falta permiso)");  
        }  
  
        if (creada2) {  
            System.out.println("Carpeta creada con éxito: " + carpetaFotos);  
        } else {  
            System.out.println("No se pudo crear la carpeta: " + carpetaFotos);  
        }  
  
        if (creada3) {  
            System.out.println("Carpeta de prueba (PUNTO 10) creada con éxito: " + carpetaPrueba);  
        } else {  
            System.out.println("No se pudo crear la carpeta: " + carpetaPrueba);  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
  
    return carpetas;  
}
```

```
Carpeta de prueba (PUNTO 10) creada con exito: pruebaTomas
```

```
No se pudo crear la carpeta: pruebaPerez
```

```
Carpeta de prueba (PUNTO 10) creada con exito: pruebaCarrillo
```

11. Renombra la carpeta pruebaUsuario por borrarUsuario.

```
// --- PUNTO 11: RENOMBRAR ---  
System.out.println("--- RENOMBRANDO CARPETA ---");  
String carpetaVieja = "prueba" + username;  
String carpetaNueva = "borrar" + username;  
  
ftp.changeWorkingDirectory( path: "/Tomas");  
  
if (ftp.rename(carpetaVieja, carpetaNueva)) {  
    System.out.println("ÉXITO: Carpeta renombrada a " + carpetaNueva);  
} else {  
    System.out.println("ERROR: No se pudo renombrar " + carpetaVieja);  
}
```

```
--- RENOMBRANDO CARPETA ---  
ÉXITO: Carpeta renombrada a borrarTomas
```

```
--- RENOMBRANDO CARPETA ---  
ERROR: No se pudo renombrar pruebaPerez
```

```
--- RENOMBRANDO CARPETA ---  
ÉXITO: Carpeta renombrada a borrarCarrillo
```

12. Intenta borrar todos los directorios creados mdUsuario, fotosUsuario y borrarUsuario.

```
System.out.println("--- BORRANDO DIRECTORIOS ---");

borrarCarpeta(ftp, carpeta: "md" + username);

borrarCarpeta(ftp, carpeta: "fotos" + username);

borrarCarpeta(ftp, carpeta: "borrar" + username);
```

```
private static void borrarCarpeta(FTPClient ftp, String carpeta) { 3 usages

    try {
        ftp.changeWorkingDirectory(path: "/tomas");

        FTPFile[] archivos = ftp.listFiles(carpeta);

        if (archivos != null && archivos.length > 0) {
            for (FTPFile f : archivos) {
                String rutaArchivo = carpeta + "/" + f.getName();
                ftp.deleteFile(rutaArchivo);
            }
        }

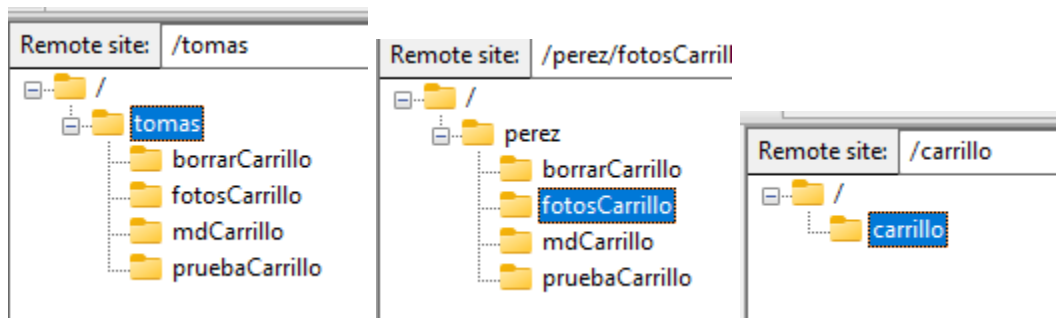
        if (ftp.removeDirectory(carpeta)) {
            System.out.println("Carpeta eliminada: " + carpeta);
        } else {
            System.out.println("No se pudo eliminar: " + carpeta);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

```
--- BORRANDO DIRECTORIOS ---  
Carpeta eliminada: mdTomas  
Carpeta eliminada: fotosTomas  
Carpeta eliminada: borrarTomas
```

```
--- BORRANDO DIRECTORIOS ---  
No se pudo eliminar: mdPerez  
No se pudo eliminar: fotosPerez  
No se pudo eliminar: borrarPerez
```

```
--- BORRANDO DIRECTORIOS ---  
No se pudo eliminar: mdCarrillo  
No se pudo eliminar: fotosCarrillo  
No se pudo eliminar: borrarCarrillo
```

13. Muestra el contenido de los directorios que queden creados.



No se si está así bien y si es lo que se pedía pero vaya dolores de cabeza me ha dado esta parte de la práctica. Igual el resultado final no es del todo bueno porque he tenido que ejecutar varias veces el código y demás e igual se ha creado algún conflicto. Aún así lo he intentado hacer lo mejor posible.

PARTE2. Correo electrónico

Utilizando la API Jakarta Mail realiza las siguientes tareas. Siempre que se envíe e-mail debe enviarse evidencias de que estos han sido enviados y han llegado realmente.

Antes de empezar he metido las dependencias en el pom.xml de angus-mail en la versión 2.0.5 como hemos hecho en clase. Además, he configurado la contraseña de aplicación de mi cuenta de google esta parte de la tarea y la he metido en la variable de entorno "MAIL_PASSWORD".

Antes de los ejercicios he creado también los siguientes métodos:

```
//metodo para obtener las propiedades
private static Properties datosServidor(){ 1 usage  new *

    Properties prop = new Properties();

    prop.put("mail.smtp.host", "smtp.gmail.com");
    prop.put("mail.smtp.auth", "true");
    prop.put("mail.smtp.starttls.enable", "true");
    prop.put("mail.smtp.port", "587");
    prop.put("mail.smtp.username", System.getenv(name: "MAIL_USER"));
    prop.put("mail.smtp.password", System.getenv(name: "MAIL_PASSWORD"));

    return prop;

}
```

```
//metodo para obtener la sesion
private static Session crearSession(Properties prop){ 1 usage  new *

    Authenticator auth = getPasswordAuthentication() → {
        return new PasswordAuthentication(System.getenv(name: "MAIL_USER"),
            System.getenv(name: "MAIL_PASSWORD"));
    };

    return Session.getInstance(prop, auth);

}
```

1. Crea un programa en Java que, utilizando la clase Scanner, solicite al usuario los siguientes datos por terminal: Destinatario (E-mail), destinatario oculto, asunto del mensaje, cuerpo del mensaje (en formato texto plano) y la ruta de un archivo local para adjuntar (opcional).

```
public static void main(String[] args) { new *
    Scanner sc = new Scanner(System.in);

    ArrayList<String> datosDestino = obtenerDatos(sc);

    try {
        ejercicio1(datosDestino);
    } catch (MessagingException | IOException e) {
        throw new RuntimeException(e);
    }
}
```

```
//metodo para obtener los datos en un ArrayList
private static ArrayList<String> obtenerDatos(Scanner sc) { 1 usage new *

    ArrayList<String> datos = new ArrayList<>();
    System.out.println("Introduce el destinatario(email):");
    datos.add(sc.nextLine());
    System.out.println("Introduce el destinatario oculto (email):");
    datos.add(sc.nextLine());
    System.out.println("Introduce el asunto del mensaje: ");
    datos.add(sc.nextLine());
    System.out.println("Introduce el cuerpo del mensaje (texto plano): ");
    datos.add(sc.nextLine());
    System.out.println("Introduce la ruta de un archivo local para adjuntar: ");
    datos.add(sc.nextLine());

    return datos;
}
```

```
private static void ejercicio1(ArrayList<String> datosDestino) throws MessagingException, IOException { 1 usage new *

    //Mapeo de los datos
    String destinatario = datosDestino.get(0);
    String destinatarioOculto = datosDestino.get(1);
    String asunto = datosDestino.get(2);
    String cuerpo = datosDestino.get(3);
    String rutaArchivo = datosDestino.get(4);

    //Obtener propiedades y sesión
    Properties prop = datosServidor();
    Session session = crearSession(prop);

    MimeMessage msg = new MimeMessage(session);
```

```
C:\Users\tomas\.jdk\openjdk-24\bin\java.exe "-javaagent
Introduce el destinatario(email):
13788476@alu.murciaeduca.es
Introduce el destinatario oculto (email):
tomasperez21@gmail.com
Introduce el asunto del mensaje:
prueba java mail
Introduce el cuerpo del mensaje (texto plano):
Hola esto es una prueba para ver si funciona
Introduce la ruta de un archivo local para adjuntar:

Process finished with exit code 0
```

prueba java mail

External

Recibidos x



deckmagick1@gmail.com

12:47 (hace 0 minutos)



para mí ▼

Hola esto es una prueba para ver si funciona

Perfectamente.

Sí, llegó.

Prueba.

← Responder

→ Reenviar

2. Crea un programa en Java, que envíe un mensaje con texto en HTML y lleve adjunto tres documentos de distinto tipo.

```
private static void ejercicio2() throws MessagingException, IOException {
    Properties prop = datosServidor();
    Session session = crearSession(prop);

    MimeMessage msg = new MimeMessage(session);

    msg.setFrom(new InternetAddress(prop.getProperty("mail.smtp.username"))); //Mi variable de entorno

    msg.setSubject("Ejercicio 2 (HTML Y 3 ADJUNTOS)");
    msg.setRecipients(Message.RecipientType.TO, InternetAddress.parse("addresslist: 13788476@alu.murciaeduca.es"));

    //cuerpo del mensaje (html) y adjuntos
    Multipart multipart = new MimeMultipart();

    MimeBodyPart parteTexto = new MimeBodyPart();

    String textoHTML = "<h1 style='color: #2980b9;'> Buenas tardes </h1>" +
        "<p> Este es un correo de <strong> prueba </strong> enviado desde mi aplicación java </p> <br>" +
        "Un saludo, <br>" +
        "Tomás Pérez";

    parteTexto.setContent(textoHTML, "text/html; charset=utf-8");
    multipart.addBodyPart(parteTexto);

    MimeBodyPart adjunto1 = new MimeBodyPart();
    adjunto1.attachFile("src/main/resources/archivosCorreo/4. Generación de servicios en red.pdf");
    multipart.addBodyPart(adjunto1);

    MimeBodyPart adjunto2 = new MimeBodyPart();
    adjunto2.attachFile("src/main/resources/archivosCorreo/prueba.txt");
    multipart.addBodyPart(adjunto2);

    MimeBodyPart adjunto3 = new MimeBodyPart();
    adjunto3.attachFile("src/main/resources/archivosCorreo/Gemini_Generated_Image_tu8iuitu8iuitu8i-removebg-preview.png");
    multipart.addBodyPart(adjunto3);

    msg.setContent(multipart);
    Transport.send(msg);
}
```

Ejercicio 2 (HTML Y 3 ADJUNTOS) External Recibidos x



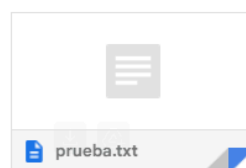
deckmagick1@gmail.com
para mí ▾

Buenas tardes

Este es un correo de **prueba** enviado desde mi aplicación java

Un saludo,
Tomás Pérez

3 archivos adjuntos • Analizados por Gmail Añadirlo todo a Drive



Los archivos los he subido a un directorio dentro de la carpeta resources.

3. Crea un programa que devuelva el listado de mensajes (10 máximo) mostrados de más reciente a más antiguo y en los que se pueda filtrar por dos valores: Asunto y Remitente. Los datos deben ser mostrados en terminal y debe mostrar al menos (se recomienda que se muestre más información)

a. Fecha de envío en formato dd/mm/yyyy hh:MM:ss à 12/03/2019 17:14:23

b. Dirección y nombre del remitente

c. Asunto

d. Las dos primeras líneas del mensaje

```
try {

    Store store = sesion.getStore( protocol: "imap");
    store.connect(prop.getProperty("mail.imap.host"),
        prop.getProperty("mail.imap.username"),
        prop.getProperty("mail.imap.password"));

    Folder carpeta = store.getFolder( s: "INBOX");
    carpeta.open(Folder.READ_ONLY);

    Message[] listaMensajes = ejercicio3(sc, carpeta);

    //Formato para la fecha
    SimpleDateFormat sdf = new SimpleDateFormat( pattern: "dd/MM/yyyy HH:mm:ss");

    for (Message mensaje : listaMensajes) {

        if (mensaje == null) continue; //Por si hay menos de 10 mensajes que el programa no reviente

        System.out.println("Fecha: " + sdf.format(mensaje.getSentDate()));
        System.out.println("Remitente: " + Arrays.toString(mensaje.getFrom()));
        System.out.println("Asunto: " + mensaje.getSubject());

        //Las dos primeras líneas
        String contenido = obtenerTextoMensaje(mensaje);
        String[] lineas = contenido.trim().split(System.lineSeparator()); //separo por espacio
        System.out.println("DOS PRIMERAS LÍNEAS DEL MENSAJE: ");

        for (int i = 0; i < 2; i++) {
            System.out.println(lineas[i].trim());
        }
    }

    carpeta.close( b: false);
    store.close();

} catch (MessagingException | IOException ex) {
    throw new RuntimeException(ex);
}
```

```
//INVESTIGACIÓN: Esta funciona me la ha hecho la IA porque hacia falta para extraer texto plano ya que los correos
// modernos suelen ser multipart...
private static String obtenerTextoMensaje(Part p) throws MessagingException, IOException { 2 usages new *
    if (p.isMimeType("text/plain")) {
        return (String) p.getContent();
    }
    if (p.isMimeType("multipart/*")) {
        Multipart mp = (Multipart) p.getContent();
        for (int i = 0; i < mp.getCount(); i++) {
            String s = obtenerTextoMensaje(mp.getBodyPart(i));
            if (s != null) return s;
        }
    }
    return "";
}
```

```
//Metodo que devuelve una lista de 10 mensajes con los filtros aplicados y ordenados de mas recientes a antiguos
private static Message[] ejercicio3(Scanner sc, Folder carpeta) throws MessagingException { 1 usage new *
    System.out.println("Introduce el asunto para filtrar: ");
    String filtroAsunto = sc.nextLine().trim().toLowerCase();

    System.out.println("Introduce el remitente para filtrar: ");
    String filtroRemitente = sc.nextLine().trim().toLowerCase();

    Message[] listaMensajes = new Message[10]; //10 mensajes (enunciado)

    int contadorMensajes = 0; //Variable para controlar los mensajes que entran a la lista

    Message[] listaCompleta = carpeta.getMessages(); //Obtengo todos los mensajes

    //Aquí se aplican los filtros y se llena la listaMensajes. Se recorre al revés para obtener los recientes
    for (int i = listaCompleta.length - 1; i >= 0 && contadorMensajes < 10; i--) {

        Message msg = listaCompleta[i];
        String asuntoMsg = msg.getSubject().toLowerCase();
        String remitenteMsg = msg.getFrom()[0].toString().toLowerCase();

        if (asuntoMsg.contains(filtroAsunto) && remitenteMsg.contains(filtroRemitente)) {
            listaMensajes[contadorMensajes] = msg;
            contadorMensajes++;
        }
    }

    return listaMensajes;
}
```

Para este ejercicio he tenido que buscar la función `obtenerTextoMensaje()` la cual recibe el objeto del mensaje y en base al tipo de contenido devuelve el texto plano o no. La función del `ejercicio3` devuelve una lista de mensajes con los filtros ya aplicados pedidos al usuario. La prueba la he hecho con el asunto "steam" y el remitente "noreply". Devuelve correctamente los 10 mensajes

ordenados por la fecha de más recientes a más antiguos como especifica el enunciado. He intentado comentar lo más relevante a lo largo del código para que sea más fácil de seguir.

Salida por terminal del programa con los filtros mencionados:

```
Introduce el asunto para filtrar:
steam
Introduce el remitente para filtrar:
noreply
Fecha: 26/01/2026 19:39:08
Remitente: [Steam Support <noreply@steampowered.com>]
Asunto: ¡Gracias por comprar en Steam!
DOS PRIMERAS LÍNEAS DEL MENSAJE:
Hola, decktomasico21:
Gracias por tu reciente transacción en Steam. Para ver los detalles, visita
Fecha: 13/01/2026 21:04:41
Remitente: [Steam Support <noreply@steampowered.com>]
Asunto: ¡Gracias por activar tu producto en Steam!
DOS PRIMERAS LÍNEAS DEL MENSAJE:
Hola, decktomasico21:
La activación de «REMATCH» se ha realizado correctamente.
Fecha: 05/01/2026 18:16:22
Remitente: [Steam Support <noreply@steampowered.com>]
Asunto: ¡Gracias por comprar en Steam!
DOS PRIMERAS LÍNEAS DEL MENSAJE:
Hola, decktomasico21:
Gracias por tu reciente transacción en Steam. Para ver los detalles, visita
Fecha: 02/01/2026 21:07:26
```

PARTE3. Correo electrónico

Usando la API de la Nasa que nos da una imagen diaria desde el 16 de junio de 1995 (https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY&date=AÑO-MES-DIA), obtén la ruta de dicha imagen (url), el título (tittle) y la explicación de esta (explanation) para la imagen de tu fecha de nacimiento. Debemos obtener el JSON de la cadena resultante y obtener los valores de las claves correspondientes que queremos (si has nacido antes del 16 de junio de 1995, obtén la imagen de algún año para tu día y mes de nacimiento). Esta información se guardará en un fichero .md en sucesivas líneas que tendrá de nombre yyyyMMdd-hhmmss-NombreApellido1Apellido2.md. Por ejemplo, 20310217-173223-GuillermoPalazonCano.md y se mostrará además su contenido por terminal.

```
public class EjercicioNasa { new *

    public static void main(String[] args) { new *

        String fechaNacimiento = "1999-08-26";

        String urlApi = "https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY&date=" + fechaNacimiento;

        try {

            HttpClient client = HttpClient.newHttpClient();
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(urlApi))
                .GET()
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

            if (response.statusCode() == 200) {

                JSONObject json = new JSONObject(response.body());

                String titulo = json.getString( key: "title");
                String urlImagen = json.getString( key: "url");
                String explicacion = json.getString( key: "explanation");

                String timestamp = new SimpleDateFormat( pattern: "yyyyMMdd-HH:mm:ss").format(new Date());
                String nombreArchivo = timestamp + "-TomasPerezCarrillo.md";

                String contenidoMd = " Título: " + titulo + "\n\n" +
                    "URL de la imagen: " + urlImagen + "\n\n" +
                    "Explicación:\n" + explicacion;

                try {

                    FileWriter writer = new FileWriter(nombreArchivo);
                    writer.write(contenidoMd);
                    System.out.println("--- CONTENIDO DEL ARCHIVO GENERADO (" + nombreArchivo + ") ---");
                    System.out.println(contenidoMd);
                    System.out.println("\nArchivo guardado correctamente.");
                } catch (IOException e){
                    System.out.println(e.getMessage());
                }

            } else {

                System.out.println("Error al conectar con la API: " + response.statusCode());
            }

        } catch (IOException | InterruptedException e) {

            e.printStackTrace();
        }

    }

}
```

Salida por terminal:

```
C:\Users\tomas\.jdk\openjdk-24\bin\java.exe "-javaagent:C:\Program Files\JetBrains\I
--- CONTENIDO DEL ARCHIVO GENERADO (20260207-164111-TomasPerezCarrillo.md) ---
Título: Cassini Flyby

URL de la imagen: https://apod.nasa.gov/apod/image/9908/qarradd\_cassini\_1.jpg

Explicación:
Connect the dots and you'll trace the path of the Cassini spacecraft as it took a fi

Archivo guardado correctamente.

Process finished with exit code 0
```

Imagen:



Explicación:

El programa consulta la API APOD de la NASA mediante HttpClient para obtener el título, la URL y la explicación de la imagen astronómica de tu fecha de nacimiento. Tras recibir el JSON, procesa los datos con la librería org.json y los vuelca simultáneamente en la terminal y en un archivo Markdown. El fichero se guarda con un nombre que combina la marca de tiempo actual y tus datos personales (yyyyMMdd-hhmmss-NombreApellido.md). Esta solución garantiza la persistencia de la información y cumple con los requisitos de integración de servicios en red del ejercicio. Antes de nada lo primero ha sido agregar las dependencias al pom.xml.

```
<dependency>  
  <groupId>org.json</groupId>  
  <artifactId>json</artifactId>  
  <version>20231013</version>  
</dependency>
```